# Lecture 33: Address Translation

CMPS 221 – Computer Organization and Design

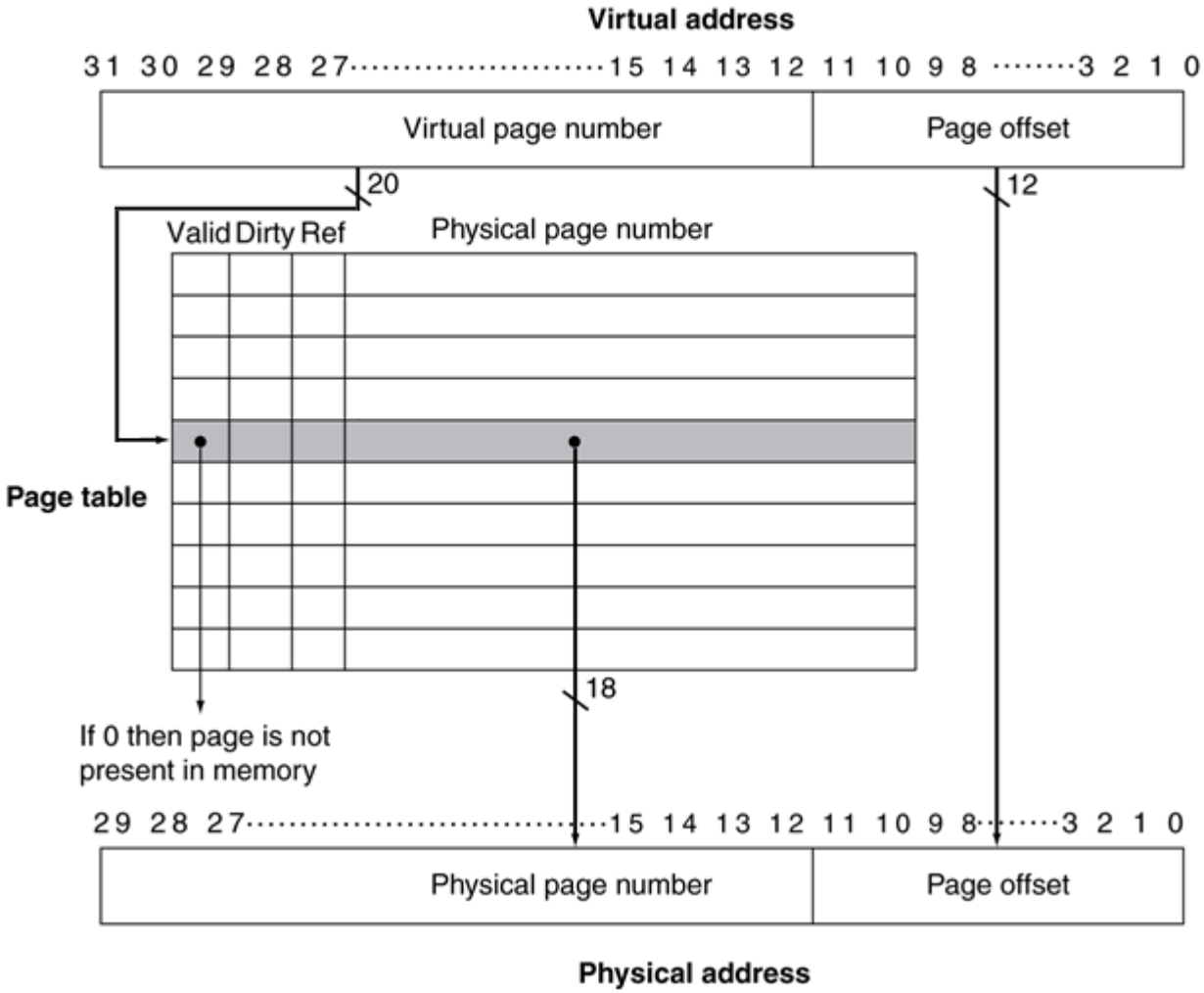
*Adapted by Izzat El Hajj from textbook slides for Computer Organization and Design, by David Patterson and John Hennessy, Morgan Kaufmann.*

# Storing Page Tables

- Page tables are large
  - Example:
    - 20-bit virtual page numbers => $2^{20}$ PTEs
    - Each PTE has an 18-bit physical page number + 1 valid bit + 1 dirty bit + 1 reference bit = 21 bits per PTE
      - Access to PTE needs to be byte aligned to a power of two
      - Use 4 bytes per PTE
    - Total page table size = $2^{20}$ * 4 bytes = 4 MiB

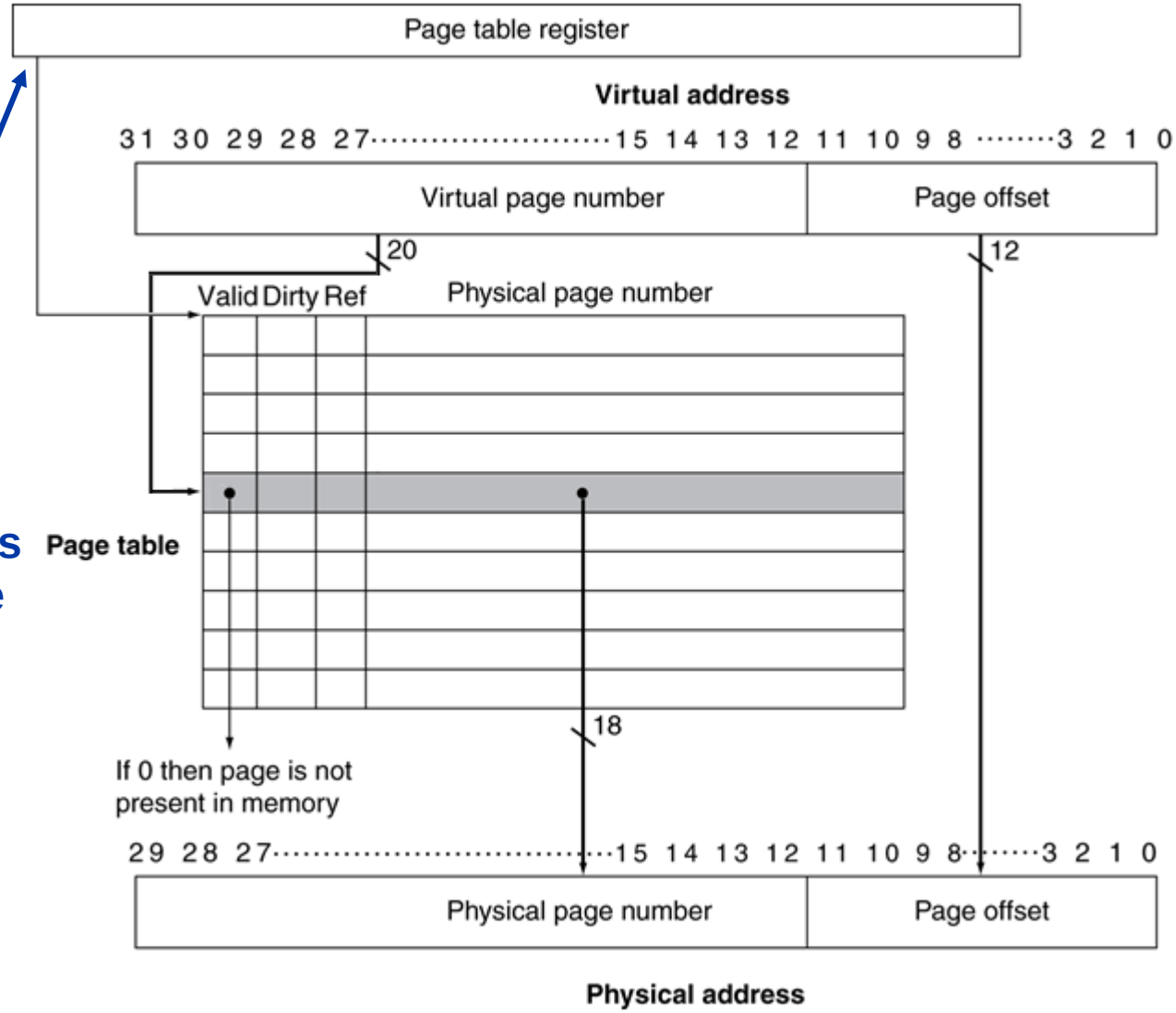
- Page tables are stored in main memory
  - To find its page table, a running program has a **page table register** in hardware that points to the beginning of the page table in physical memory

# Finding the Page Table



# Finding the Page Table

**Page table register points to page table in physical memory**



# Accessing Page Tables

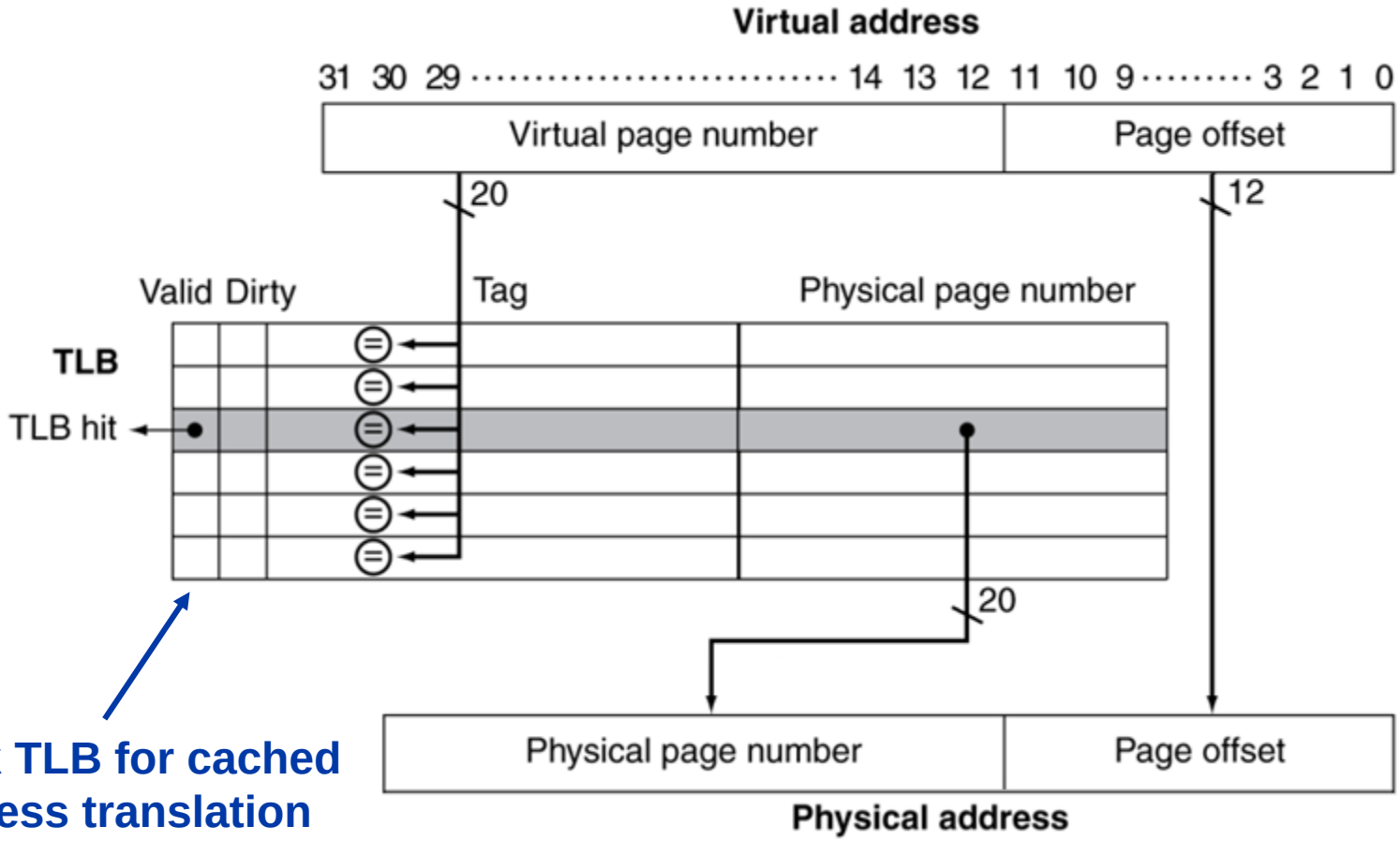
- Every memory access becomes two accesses
  - One to access the page table for address translation
    - Typically more for compact page table representations
  - One to access the actual memory

- Solution: use a special cache for translation

# Translation Lookaside Buffer

- The **translation lookaside buffer** (TLB) is a special cache for PTEs
- Does not need to be large => low hit time
  - Every page (e.g., 4KiB) of memory needs only 1 PTE
  - Page table accesses have very good locality
  - Typical size: 16–512 PTEs
  - Typical time: 0.5–1 cycle for a hit
- High associativity => low miss rate
  - Typical associativity: fully associative
  - Typical miss rate: 0.01%–1% miss rate

# Fast Translation with TLBs




**Check TLB for cached address translation**
(read from page table in memory only if TLB misses)

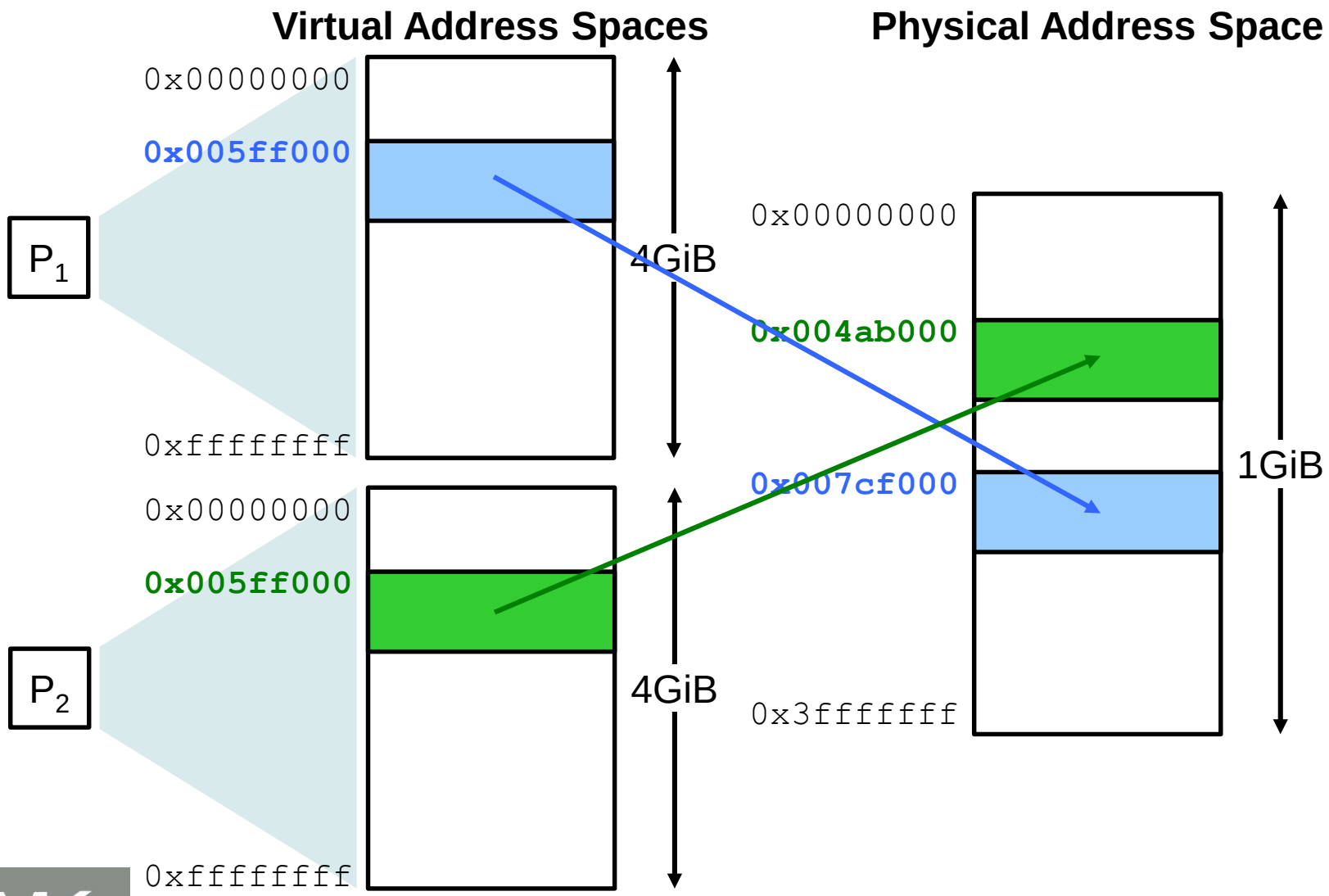
# TLB Misses and Page Faults

- If the TLB valid bit is 1, there is a **TLB hit** so read the translation from the TLB
- If the TLB valid bit is 0, there is a **TLB miss** so try to read the translation from the page table in memory
  - If the PTE valid bit is 1, the page is in memory so load the translation to the TLB
  - If the PTE valid bit is 0, the page is not in memory (**page fault**) so fetch it from disk
- Note: the valid bits in the TLB and the PTE have different meanings!

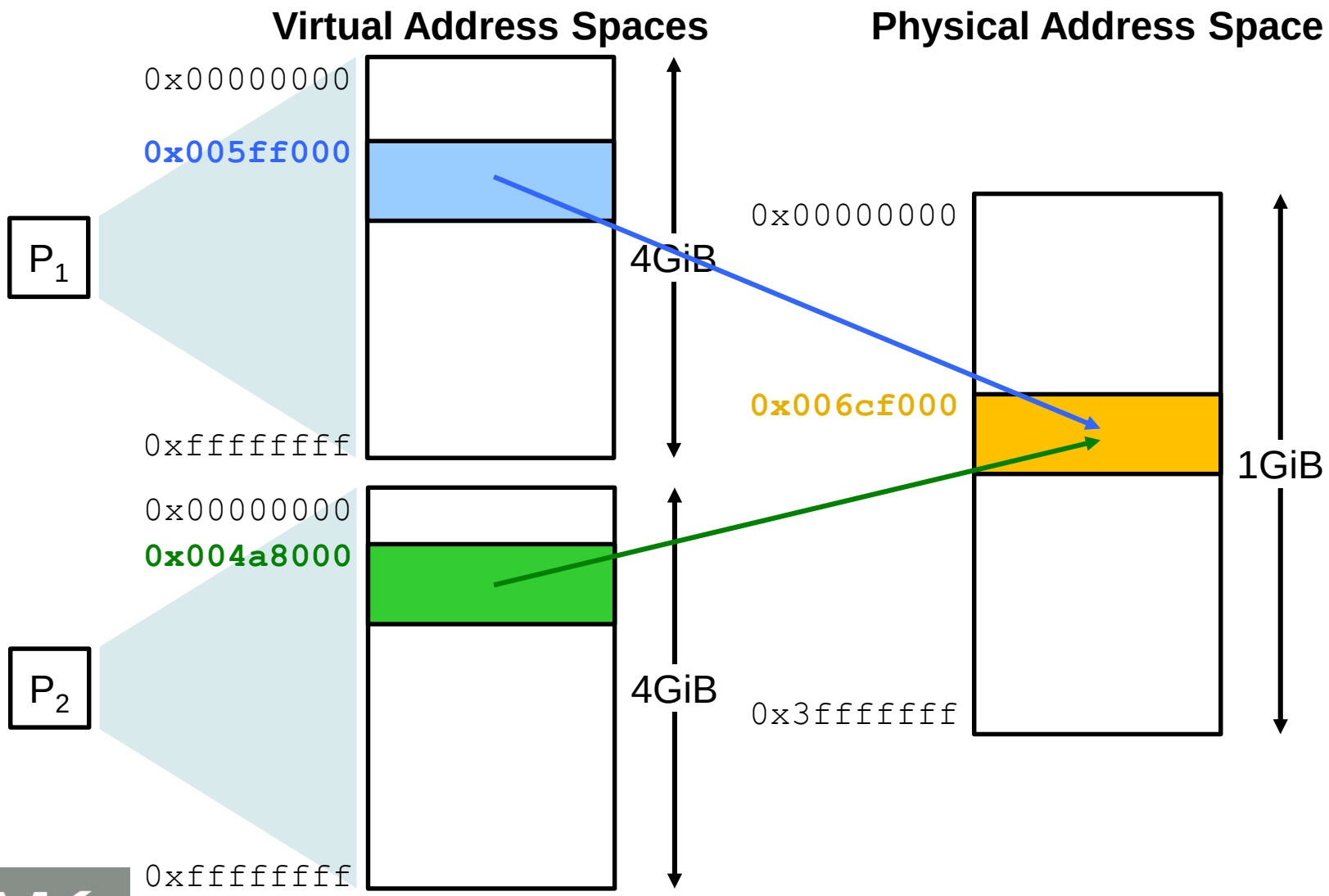
# Translation in the Hierarchy

- Where does translation take place in the memory hierarchy?
  - Typically, translation occurs before L1 cache
  - Otherwise, accessing the L1 cache with virtual addresses causes several problems:
    - **Homonym problem**: the same virtual address from different processes map to different physical addresses
    - **Synonym problem**: different virtual addresses from different processes map to the same physical address
    - We would need to empty the cache every time we switch processes

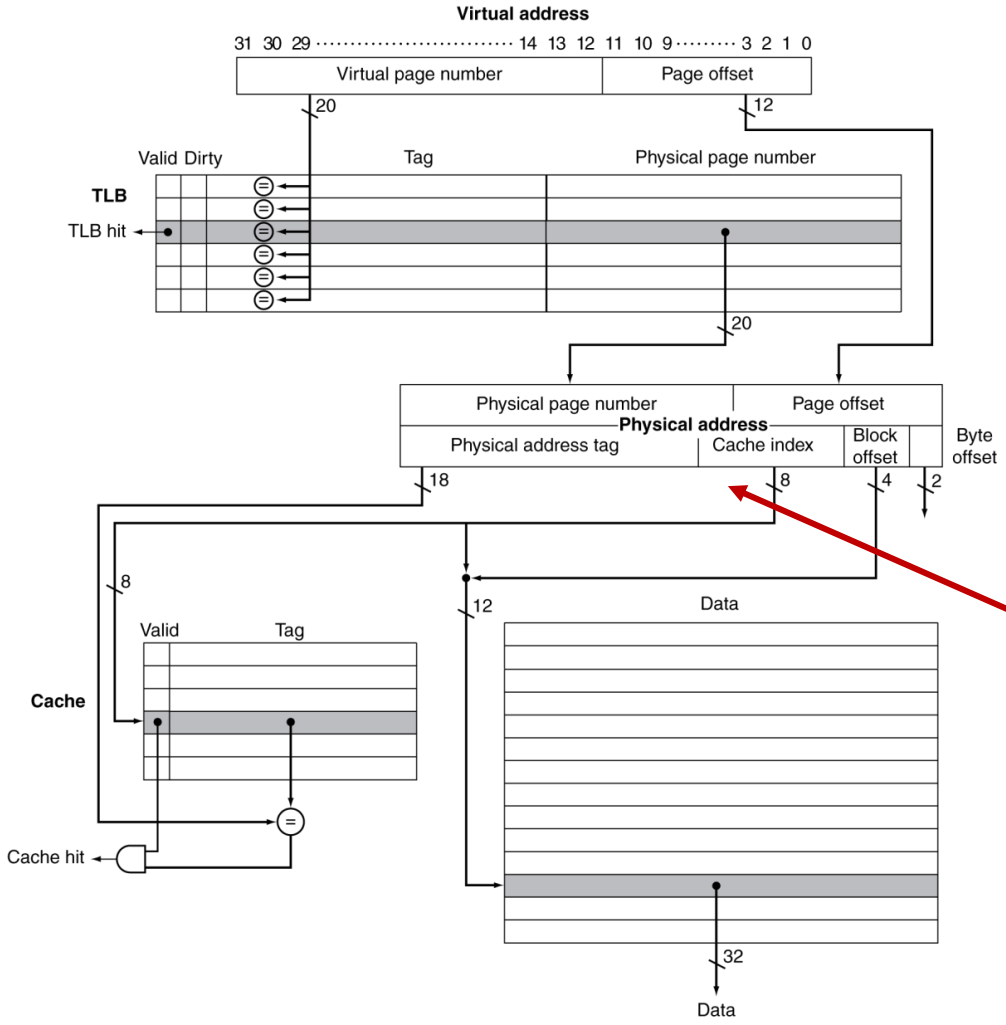
# Homonym Problem



# Synonym Problem

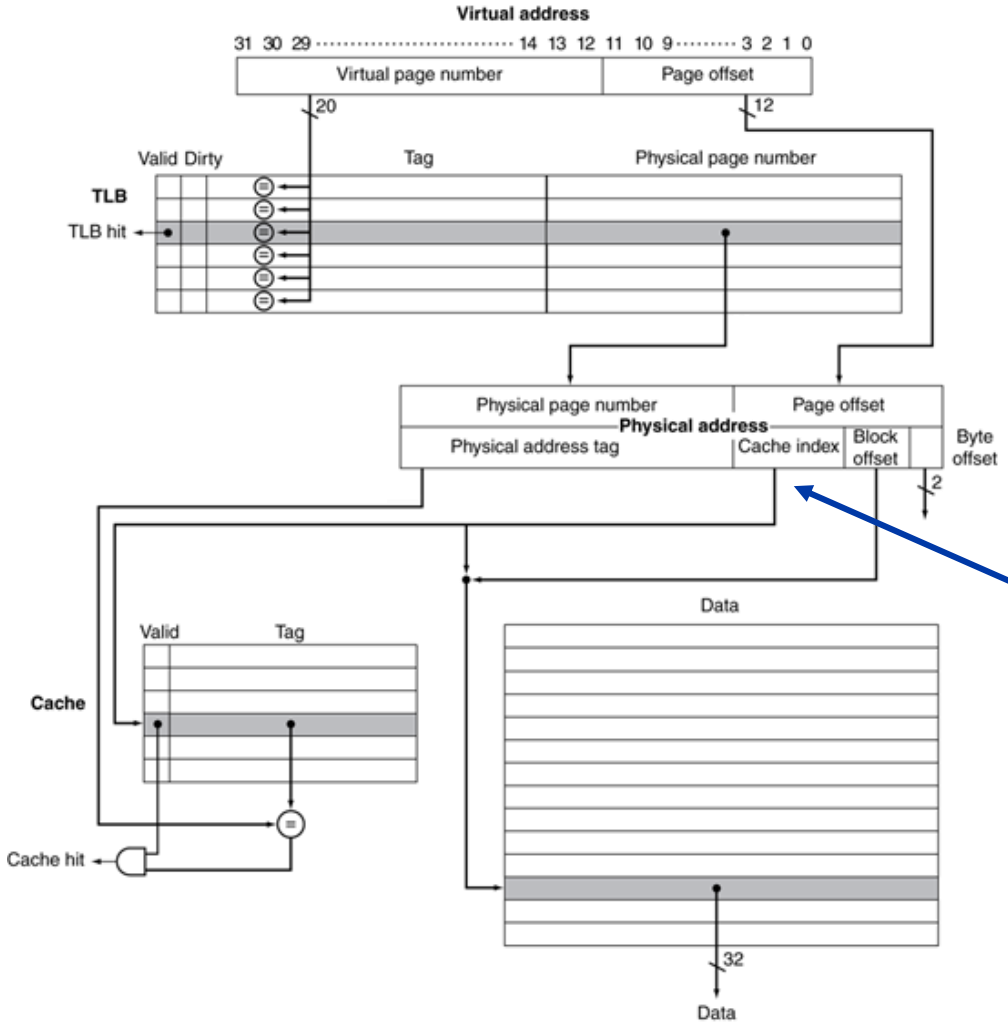


# TLB and Cache Interaction



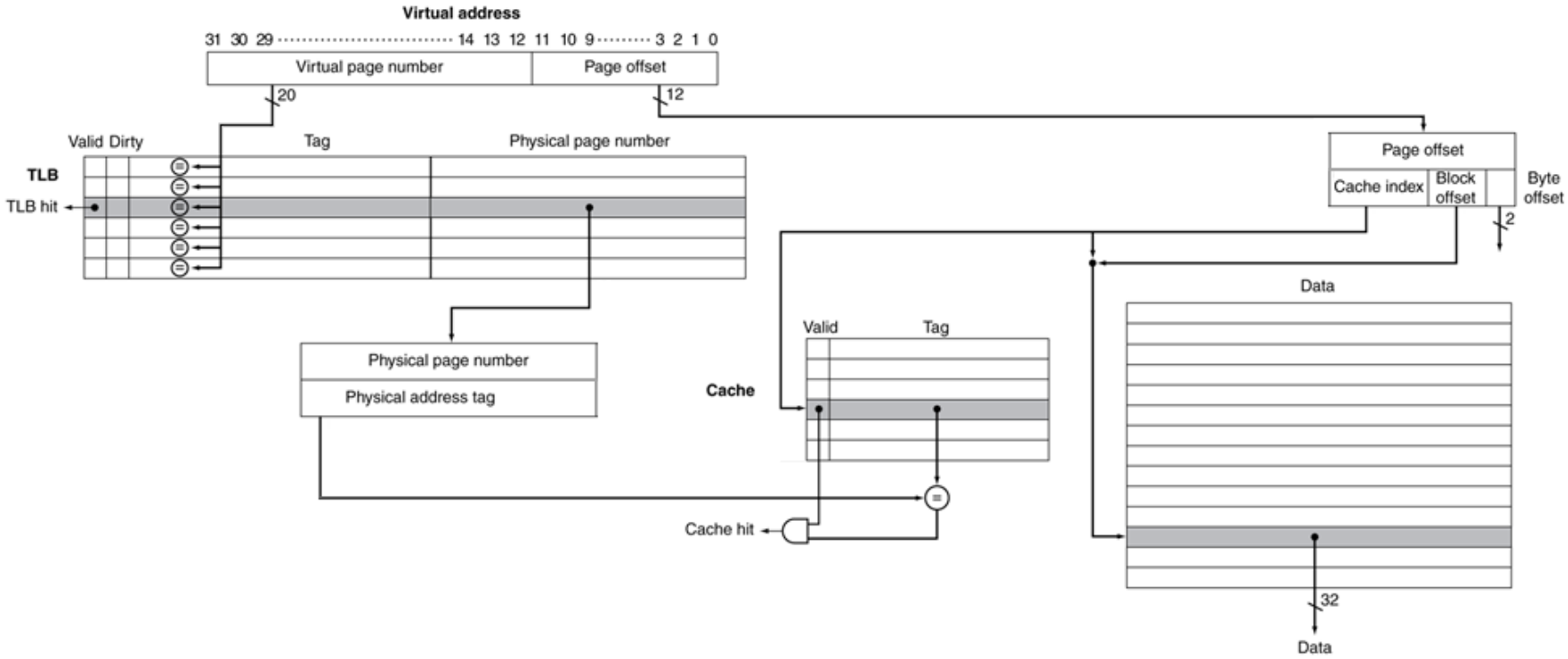

**Since index uses part of the PPN, we need to wait to finish translation before accessing the cache**

# TLB and Cache Interaction




**If we size the cache such that the index only comes from the page offset, we can index the cache and translate the tag in parallel**

# TLB and Cache Interaction




A **virtually-indexed physically tagged cache** extracts the index from the virtual address to index the cache in parallel while the tag is being translated

# Textbook Sections

- The content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 5.7